Report: Diffusion Models and Logistic-Map-Based Noising

Abhijeet Vikram

December 3, 2025

Abstract

This report presents an overview of the forward diffusion noising process used in Denoising Diffusion Probabilistic Models (DDPMs), outlining the mathematical foundations and the reconstruction of the diffusion model as part of this project. I then introduce a modification to the standard diffusion framework in which the Gaussian noise schedule is replaced with a deterministic chaotic process based on the logistic map. The aim of this substitution is to investigate whether controlled chaos can serve as an alternative to randomness in the noising mechanism. I describe the formulation of the logistic-based diffusion process, analyze its theoretical properties, and present experimental results in the hope of achieving computational efficiency over DDPM models.

1 Introduction

Diffusion models are generative models that gradually transform a data sample into noise using a fixed forward "noising" process, and then learn the reverse transformation using a neural network. The forward process is mathematically tractable and typically involves adding Gaussian noise with variance controlled by a schedule. In this report, we first summarize the classical diffusion formulation and then describe a modification where the noise is produced using the logistic map at its chaotic parameter.

2 Classical Diffusion Forward Process

The forward diffusion process is defined as a Markov chain that progressively adds noise to the data. Given a clean sample x_0, the forward transition is:

$$x_t = \sqrt{\beta_t}\, x_{t-1} \; + \; \sqrt{1-\beta_t}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I), \tag{1}$$

so, probability q of getting x_t given x_{t-1}

$$q(x_t \mid x_{t-1}) = \mathcal{N}\Big(x_t; \sqrt{1-\beta_t}\, x_{t-1},\, \beta_t I\Big)\,, \tag{2}$$

where β_t is a small variance term controlling the amount of noise injected at timestep t. By composing all t steps, we obtain a closed-form expression:

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 \; + \; \sqrt{1-\bar{\alpha}_t}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I), \tag{3}$$

where

$$\bar{\alpha}_t = \prod_{s=1}^{t} (1-\beta_s). \tag{4}$$

therefore,

$$q(x_t \mid x_0) = \mathcal{N}\big(x_t; \sqrt{\bar{\alpha}_t}\, x_0,\, \sqrt{1-\bar{\alpha}_t} I\big)\,, \tag{5}$$

This expression allows the model to train on arbitrary timesteps without explicitly simulating the entire chain.

2.1 Training Objective

The core training task is to predict the noise that was added. The model learns a function $\epsilon_\theta(x_t, t)$ and is optimized via:

$$\mathcal{L}(\theta) = \mathbb{E}_{x_0, t, \epsilon}\left[\|\epsilon - \epsilon_\theta(x_t, t)\|^2\right]. \tag{6}$$

This objective is equivalent to maximizing a variational lower bound and has been shown to yield strong generative performance.

3 Reverse Process

During sampling the reverse process in diffusion models reconstructs the sample by iteratively denoising:

$$P(A \mid B, C) = \frac{P(B \mid A, C)P(A \mid C)}{P(B \mid C)} \tag{7}$$

$$q(x_{t-1} \mid x_t, x_0) = \frac{q(x_t \mid x_{t-1}, x_0)q(x_{t-1} \mid x_0)}{q(x_t \mid x_0)} \tag{8}$$

by the markovian nature of the forward process this can be written as:

$$q(x_{t-1} \mid x_t, x_0) = \frac{\mathcal{N}\big(x_t; \sqrt{\alpha_t}x_{t-1}, (1-\alpha_t)I\big)\,\mathcal{N}(x_{t-1}; \sqrt{\bar{\alpha}_{t-1}}x_0, (1-\bar{\alpha}_{t-1})I)}{\mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)I)} \tag{9}$$

$$q(x_{t-1} \mid x_t) = \mathcal{N}\big(x_{t-1}; \mu_q(x_t, x_0), \sigma_q^2(t)I\big)\,, \tag{10}$$

where the mean depends on the network's noise prediction:

$$\mu_q(x_t, x_0) = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\bar{\epsilon}_t\right), \qquad \sigma_q^2(t) = \frac{(1-\alpha_t)(1-\bar{\alpha}_{t-1})}{(1-\bar{\alpha}_t)} \tag{11}$$

here ϵ_t is the noise that drove the transition from x_0 to x_t. To deal with this, we will train a neural network to predict ϵ_t. After training, the network will be a black box which takes a value of t and a noisy image x_t as input and returns a prediction of $\bar{\epsilon}_t$ as ϵ_θ so we can get x_{t-1} by:

$$x_{t-1} = \frac{1}{\sqrt{\bar{\alpha}_t}}\left(x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta\right) + \sigma_q(t)Z, \qquad Z \sim \mathcal{N}(0, I) \tag{12}$$

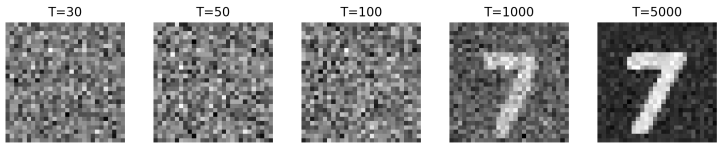

Figure 1: Denoising process in Diffusion model for U-NET model trained on MNIST images of 7.

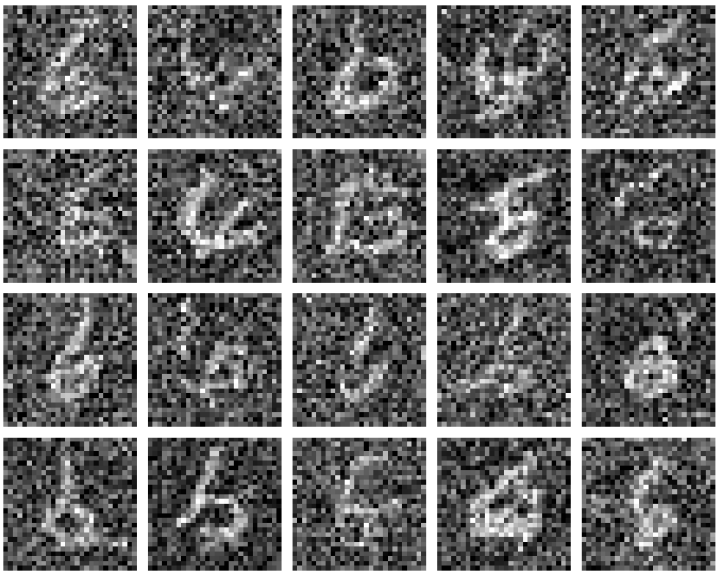

Figure 2: Denoising process can be conditioned over some input value or vector, 20 different outputs for U-NET model trained on complete MNIST dataset and image generated for label input 6.

4 Sampling or Generation

During sampling, an image of the similar shape as training images is generated completely randomly from a $Normal(0, 1)$ distribution.

$$x_t = I, \qquad I \sim \mathcal{N}(0, 1) \tag{13}$$

This image is used as x_t and with t is given to the trained model which gives a prediction ϵ_θ for $\bar{\epsilon_t}$ by which we calculate x_{t-1} by using eq.(12) iteratively and reach to x_0, the original image associated with the random image.

5 Logistic-Map-Based Noising

5.1 Motivation

Instead of injecting Gaussian random noise, one may replace it with a deterministic chaotic process. The logistic map at parameter $r = 4$ is:

$$x_{n+1} = 4x_n(1 - x_n), \tag{14}$$

is known to produce chaotic, non-repeating values in the interval $(0, 1)$ for almost all initial conditions. This when used for noising would be deterministic and hopefully easier for the model to learn than a stochastic diffusion process.

5.2 Constructing Logistic Noise

Let i_0 denote the random image chosen from the image dataset, we begin the noising process by applying logistic eq.(14) iteratively:

$$i_t = 4i_{t-1}(1 - i_{t-1}), \qquad i_0 \in (0, 1) \tag{15}$$

this can be written as,

$$i_t = \mathcal{N}(i) + i_0 \tag{16}$$

therefore, total noise added is,

$$\mathcal{N}(i) = i_t - i_0 \tag{17}$$

Here i_t is known to follow the $beta(0.5, 0.5)$ PDF at $\theta = 4$, therefore $\mathcal{N}(i)$ will also follow a shifted $beta(0.5, 0.5)$ distribution, since i_0 is a constant. This shows that the noise follows a closed form probability distribution which could be learnt by a model theoretically.

Now, this image i_t, with t is entered into the model and trained to predict the amount of noise added to the image, or to denoise the image to i' directly. using,

$$\mathcal{L}(\theta) = \mathbb{E}_{x_0,t,\epsilon}\left[\|\epsilon - \epsilon_\theta(x_t, t)\|^2\right]. \tag{18}$$

$$\mathcal{L}(\theta) = \mathbb{E}_{x_0,t,i}\left[\|i_0 - i'(x_t, t)\|^2\right]. \tag{19}$$

eq.19 was done in the hope of gaining compute advantage over current diffusion models by avoiding the denoising process completely. This was based upon the hypothesis that deterministic noise would be much easier for the model to learn and remove than a stochastic noise.

5.3 Sampling

z_t in the logistic equation is also shown to follow a $beta(0.5, 0.5)$ probability distribution function as shown in img.

$$z_t \sim beta(0.5, 0.5) \tag{20}$$

During sampling, a new image is generated randomly from (20) then entered

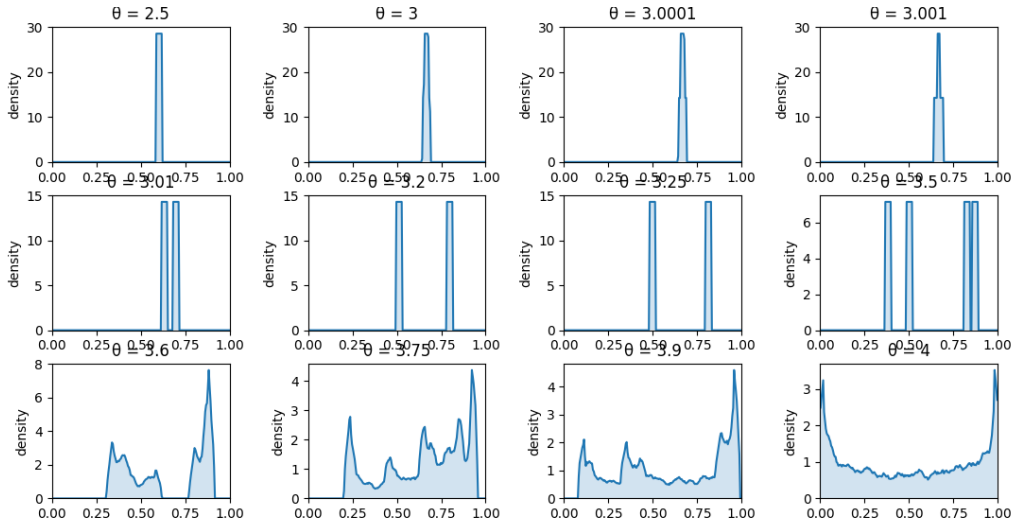

Figure 3: Probability distribution function for different x_0 for different θ. Notice U-shaped $beta(0.5, 0.5)$ distribution for $\theta = 4$, chaos.

to the trained model which gives output the amount of noise added or the denoised image itself.

5.4 Results for MNIST Dataset

MNIST is a dataset of handwritten digits 0-9 images of 28×28 pixels and labels. These are the results obtained from different models and there inferences.

5.4.1 CNNs - Convolutional Neural Network

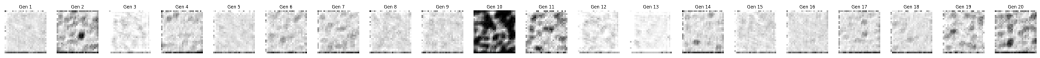

Figure 4: CNN is used to learn features in the dataset, as we are entering a completly noisy dataset it is expected for the model to learn something noisy.

5.4.2 U-NET

The model loss value was not converging and was giving a noisy outputs as U-NET model is also based of multiple CNN and MAXPOOL layers stacked which basically look for features in the images and since the input was a completely noisy image models couldn't learn anything.

5.4.3 FFN - Feed Forward Network

Surprisingly, Vanilla Feed Forward Network gave good results when trained on images of particular label here 7.

Figure 5: Denoising process in FFN after 10 steps of denoising.

Figure 6: Denoising process in FFN after 1000 steps of denoising.

Notice the 20 outputs don't capture the variations in handwriting styles as in the original MNIST dataset. Model is learning some central value for the all the images of the dataset.

5.4.4 Random Forest

So I also tried the random forest of of curiosity to see the results and achieved similar results as the FFN.

Figure 7: Actual amount of variation in styles of the handwriting in the MNIST dataset.

Figure 8: Denoising process in FFN after 1000 steps of denoising.

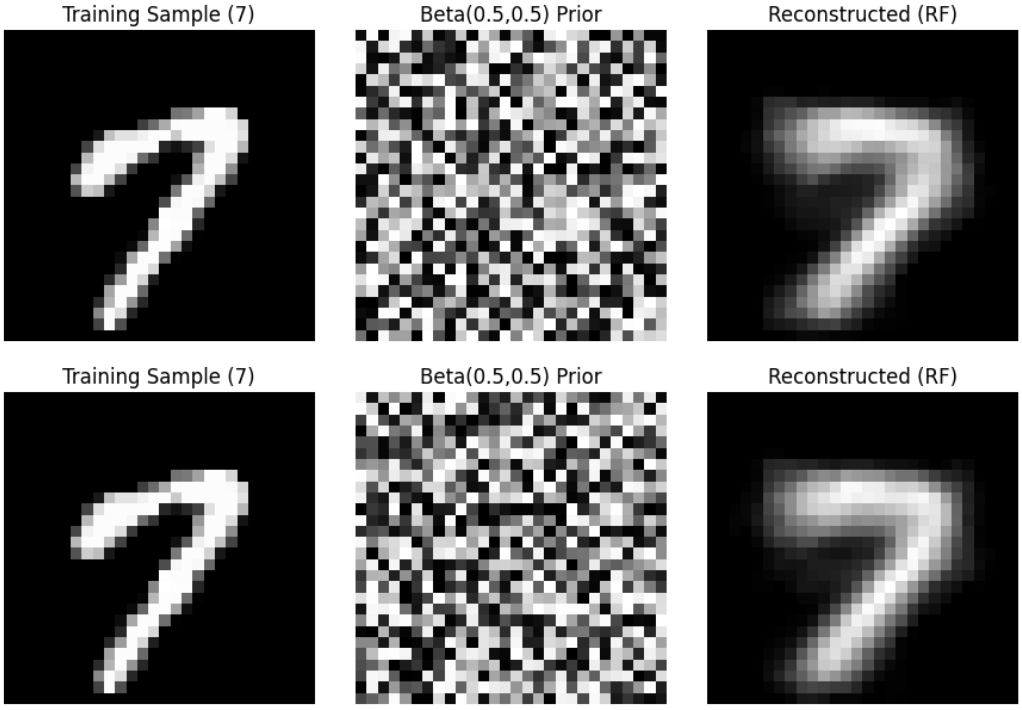

Figure 9: Random forest generated images. Notice there are pixel level changes and some style change but not in handwriting styles as in actual MNIST dataset.

5.4.5 RNN

I also tried out RNNs but the shortcomings of RNN itself of "Vanishing Gradient" overpowered since the significant part of the values in the output are close to 0 as shown in fig.3 so the loss values went to Nan.

5.4.6 VAE - Variational Autoencoder

VAE has been already shown to perform better and capture variations in the images by introducing randomness in the latent space. It first reduces the dimensions of the image progressively to a predefined latent space than a same size vector is generated by sampling over the latent space vector (A source of stochastic noise) and then again increases the dimensionality via decoder.

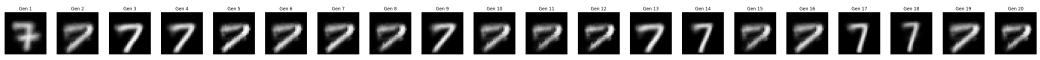

Figure 10: Notice some variations of handwriting captured by VAE by introducing some randomness in the latent space.

5.5 Results for CIFAR10 Dataset

CIFAR10 is image dataset of different classes of 32×32 pixels labeled images of cats, dogs, e.t.c. Its a significant task for the model to learn, I tried some primitive models on the image so cats.

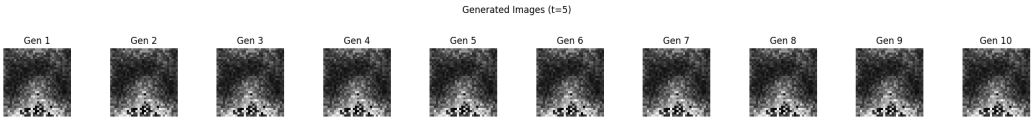

Figure 11: Cat image on VAE.

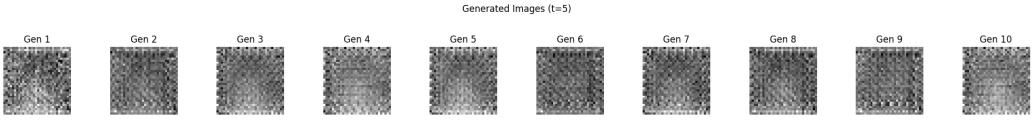

Figure 12: Cat images on VAE with Decoder with CNNs.

Notice some learning but not significant enough for the features of the cat to be visible.

6 Future Works

Future works on introducing logistic noise in the latent space of the VAE.

$$x_{t-1} = \frac{-\theta \pm \sqrt{\theta^2 - 4\theta x_t}}{2\theta} \tag{21}$$

Also notice that logistic equation is a quadratic equation and in reverse for x_{t-1} has 2 solutions and for t time steps will have 2^t solutions and optimizing algorithm to convert the binary tree produced to a vector and the path vector to the actual x_0 be learnt by a transformer architecture, which is known to show good results in learning vectors.

7 Conclusion

I reviewed the classical Gaussian diffusion noising process and presented your variation using logistic-map-based chaotic noise. While Gaussian noise yields simple closed-form training objectives and stable reverse diffusion behavior, chaotic noise gives out central values of images repetitively and could not capture the randomness and creativity in image styles and required some source of randomness as provided by VAE's the model showed variations in the output images styles. So Logistic Noising could be used as a source but to capture the variations in images styles we need some source of randomness in any form.